

Project Assignment

Start Assignment

- Due 4 Oct by 23:59
- Points 10
- Submitting a website url or a file upload
- Available 14 Sep at 13:30 - 4 Oct at 23:59

Decentralized Digital Identity and Data Sharing Platform

Problem Statement: Centralized platforms control user data and decide who can access it. This creates multiple challenges such as:

- Users have no visibility into who accesses their data
- Data breaches compromise all users simultaneously
- Users cannot easily revoke access
- No immutable audit trail of access events
- Users cannot control data sharing across platforms
- Platforms profit from user data without sharing benefits

Your team's task: Build a blockchain-based decentralized identity and data sharing platform where:

- Users control their digital identity and data
- All data access is explicitly logged and auditable
- Users grant time-limited, revocable consent
- Data requesters can only access with valid consent
- Users are incentivized via access tokens
- All actions are immutable and verifiable
- Data ownership remains with the user (tokens grant access only)

Scope/Application Domain Selection: You are free to choose the application domain for the data being shared, as this will help guide your project's requirements. Possible domains include but are not limited to:

- **Healthcare:** Patients share medical records with doctors, hospitals, or researchers. Example identity attributes: patient name, email, medical ID, blood type (or equivalent).
- **Finance:** Users demonstrate creditworthiness to lenders without disclosing full financial history. Example identity attributes: user name, email, account number, credit tier.
- **Education:** Users share digital credentials, diplomas, or certificates for employment or

institutional verification. Example identity attributes: user name, email, student/alumni ID, degree earned.

- **IoT:** Device owners grant access to sensor data for third-party applications or services. Example identity attributes: device owner name, device ID, device type, location.
- ..

Project Steps

Ensure you complete the following steps (you can add more if your team is enthusiastic):

Step 1: Research and Planning

- **Gather Information:** Investigate how current platforms handle digital identity and data sharing in your chosen application domain. Examples: healthcare portals (patient access to records), financial services (credential verification), educational credentialing systems. Document what has been done and what works and what doesn't in these systems.
- **Define Users and Roles:** Identify the actors in your platform. For example::
 - Users (identity owners who share data)
 - Requesters (doctors, researchers, lenders, employers, services that access data)
 - Possibly administrators (those who manage the system)

Map out who initiates actions and what permissions each role has.

- **Outline Functional Requirements:** Your platform must support. For example:
 - User registration with hashed identity attributes
 - Consent grants that specify duration and data type
 - Consent revocation by users
 - Data access logging (both successful and failed attempts)
 - Token rewards for sharing consent

List any additional features your domain requires.

Deliverable

Problem statement. Defined user roles, the functional requirements and a high level overview of how users interact in your systems.

Step 2: Platform/System Design

A. Design the data model

Define what attributes/data user stores. For your chosen domain, decide:

- What identity attributes are necessary? (name, email, ID, etc.)

- What gets hashed and stored on-chain?
- What stays off-chain?

Create a table showing: Attribute → Stored On-Chain? → Stored Off-Chain? → Hashed?

Design the consent model

Specify:

- What types of data can be shared in your selected domain?
- How long can consent last? (1-365 days)
- Who can grant and revoke consent?
- What happens when consent expires?

Document consent workflows as diagrams or pseudocode.

Design the audit log

Decide what events to record:

- Who accessed what data?
- When did access occur?
- Was access granted or denied?
- Can logs be deleted? (No)

B. Smart Contract Design

Identify the details of the smart contracts. Please think of the following components (A component may have more than one smart contract)

Digital Identity: This component handles user registration, stores user attributes, and considers the overall security (For example it can use hash functions for sensitive data.)

Key Functions:

- **Register User:** Require minimal data for registration. (e.g., unique ID, email).
- **Retrieve User Info:** Allow querying specific attributes via hashed values or references to off-chain data.
- **SetConsent:** Enable identity owners to create and manage custom consent agreements (e.g., enable patients to create and manage custom consent agreements specifying which doctors can access which medical records and for how long), Also, users receive tokens when they grant consent (e.g., only contract owner can call, no tokens transferred during data access, tokens only enable access, data ownership never transfers).
- **Revoke Consent:** Allow users to revoke consent granted to other users (logic is similar as mentioned in SetConsent).

- **Log Access:** Maintain an audit trail for access to off-chain data.

Data Sharing : This component enables data sharing. Data remains with the user. Smart contracts only manage permissions and track access.

Key Functions:

- **Share data:** User stores actual data locally (JSON file, text file, spreadsheet (this is your team choice and based on the decided scope)). Smart contracts store only hashes and permission records on-chain
- **Access data:** When a requester wants access, the contract checks: "Does this user have valid consent?"
- **Update Log:** The contract logs who accessed what and when

Also, tokens reward the user for sharing, but access is controlled by consent agreements only.

Deliverable

Architecture diagram (UML or others) showing your chosen design and how the components of the system interact with one another

Step 3: Implementation

Implement the selected design. Use Solidity for Smart Contracts. If possible, prefer python for any additional components surrounding smart contracts. When justified, other programming languages are possible. In terms of deployment, set up hardhat (as we used in tutorials/labs) and deploy, and test locally (no need for testnet).

Deliverable

Solidity implementation of the smart contracts with their functionalities. Implementation of any other modules that complement the functionality of smart-contracts and blockchain.

Step 4: Testing

- **Unit Testing:** Write unit tests in Solidity, as we did in LAB 3. Use Hardhat to compile and run these Solidity-based tests. Each test file should be named according to the contract it verifies (for example, DigitalIdentity.t.sol, ConsentManager.t.sol, or DataSharing.t.sol). Also, in your report, briefly explain for each tested functionality why it is critical for the overall platform design.
- **Efficient DataSharing.** Since frequent data access is expected, measure and report gas usage for key functions. Optimise contract logic where appropriate (for example, reduce storage writes, limit redundant computations). Include a table summarising deployment cost and the average gas used per function execution.

- **Integration Tests:** Test user workflows (e.g., registration → grant consent → data access → revoke consent) and verify that the expected outcomes occur.
-
- **Note:** Ensure that no personal data are used in testing the solution

Deliverable

Tables reporting all successfully performed tests and tables showing the deployment costs and cost for executing each function.

Step 5: Deployment

- **Deploy Contracts:** Deploy the smart contracts to a test network (use the methods as explained in the labs). Testing and interaction should primarily be performed in this local environment. Deployment to a public test network is optional and may be used only if the team wishes to demonstrate the system in a testnet setting.
- **Simulation and Evaluation**
 - After deployment, simulate interactions between multiple users with different roles (for example, user, requester, and administrator).
- **Record and analyse:**
 - The average gas cost required for core operations (e.g., registration, consent creation, data access).
 - The execution time for transaction confirmation and event logging if deployed in a test network.
 - If a team chooses to deploy to a public test network, compare these values with those from the local setup.
- **Front-End (Optional)**
 - If the project includes a front-end component, deploy the user interface locally or on a simple hosting service.
 - Ensure that it connects correctly to the locally deployed contracts through the appropriate RPC endpoint.

Deliverable

Tables reporting how well your solution scales in time and cost (for the amount of users that you created).

Step 6: Front-End Development (Bonus Points)

- **User Interface Design:** If your team chooses to develop a user interface, create basic wireframes to outline the structure of your application.
 - A minimal interface should allow users to:

- Register and manage their digital identity
- Create and revoke consent agreements
- View audit logs of data-sharing activities
- Create wireframes for the application layout.
- Develop a responsive UI allowing users to register, manage identities, set up consent agreements, and view audit logs.
- **Integrate with the Smart Contracts:**
 - Connect the interface to your deployed smart contracts using Viem.js (as supported in the current Hardhat environment). Please note that all interactions should occur with the local Hardhat network.
 - Ensure that key functionalities such as registration, consent management, and data access execute correctly through the interface.

Deliverable

If implemented, provide a simple front end (local deployment is sufficient) demonstrating interaction with your smart contracts via Viem.js and the local Hardhat network.

Step 7: Report and Documentation

- **Documentation:** Create a detailed report summarizing the design, architecture, and functionalities of the platform. (Check coursebook for requirements on structure and max length)
- **Instructions:** Include clear and concise instructions on how to compile, deploy, and test your code. Focus on clarity and reproducibility so that another user can easily run your solution on a local Hardhat network (or testnet).
- Prepare a short **presentation** following the format outlined in **Week 6's Lab**. The presentation should highlight the main design aspects, implementation results, and testing outcomes.

Final Deliverable

A report including all the previous deliverables. A presentation and the documented code.